

Rubric

STYLE: Check that program has correct structure and conforms with style guidelines:

(1) Program file header. Does the program include a file header with project identification, author(s), credits, and short file description at the top of the program file? Does each function include a docstring with a brief description of the function and an example of function use?

(2) Readable. Is the program readable? Are parts of the program and individual functions clearly labeled? Are functions a reasonable length (not too long)? Does each function have a return statement? Are identifiers meaningful? Are helpful in-line comments provided? Is whitespace used within expressions and between lines of code to further function readability? Could another student in the class figure the program out?

CORRECTNESS: Check that program runs and returns correct results per project specification:

(3) Project specification. Does the program meet project specifications? Do individual functions meet project specifications? Did the programmer show good judgement in deciding and documenting how ambiguous parts of the problem specification are implemented?

(4) Algorithms. Are the underlying algorithms correct and clear?

(5) Program design. Does the program use auxiliary functions appropriately? Does the program solve the problem in a reasonable manner?

(6) Program works. Does the program run? Does it return correct results for specified input values? Does it return correct results for other input values?

20 points are possible. This section describes the maximum point value for each applicable section.

Task 1 is worth a total of **9** points.

Proxy server is able to respond to a GET request from a client: **2** point

Proxy server is able to GET an index webpage: **3** points
(e.g. wikipedia.org)

Proxy server is able to give the response from a webpage to the client: **3** points

Proxy server is able to GET an arbitrary page: **1** points
(e.g. en.wikipedia.org/wiki/Computer_security)

Task 2 is worth a total of **9** points.

Program has support for error messages and they work: **2** points.

301 (not bypassing the proxy): **0.5** points

Handling other errors (404 at minimum): **1.5 points**

Program has support for POST requests and it works: **2** points.

Program successfully caches objects: **2** points.

Program successfully **returns** the **cached object** to the client **3** Points

Style is worth **2** points.

Program has additional code comments, functions have docstrings/comments. Good coding style practices and procedures.

A README may be included with sufficient instructions for running and testing the program.

Some additional guidance and specification is provided below.

19-20 points: Program is outstanding: program runs and returns correct results per project specification; program is easy to read, has additional comments detailing various parts. If the program used functions, they all have descriptive doc-strings. This program could be used as example solution.

16-18 points: Program is substantially correct according to the rubric: program runs and returns mostly correct results per project specification; comments and docstrings are present; some suggestions for minor improvements.

14-15 points: Program is mostly correct according to the rubric: program runs and returns mostly correct results per project specification; comments and docstrings are present; some suggestions for small improvements.

10-13 points: Program reflects good effort but is somewhat lacking compared to class standards. For example, there may be multiple bugs in the code, the submission may not meet one or more of the project specifications, or has other, similar, issues.

6-10 pts. Program is seriously lacking with respect to class standards. For example, the program contains many bugs, the program lacks hallmarks of good programming style, the program does not meet the project specification.

0-5 pts. -Program is even more seriously lacking with respect to class standards. For example, the program does not run at all (syntax errors).